Revisiting Setoid Models of Type Theory

Loïc Pujet

13 november 2025

I.

The setoid model

An Embarrassing Question

"How do I rewrite an expression deep inside the goal?"

```
f : A -> B
g : B -> A
Hs : forall x : B, f (g x) = x
============================
foo (fun x : B => f (g x))
```

An Embarrassing Question

"How do I rewrite an expression deep inside the goal?"

```
f : A -> B
g : B -> A
Hs : forall x : B, f (g x) = x
============================
foo (fun x : B => f (g x))
```

→ If the expression depends on bound variables, you can't

An Embarrassing Question

"How do I rewrite an expression deep inside the goal?"

```
f : A -> B
g : B -> A
Hs : forall x : B, f (g x) = x
============================
foo (fun x : B => f (g x))
```

→ If the expression depends on bound variables, you can't

...or, well, you can always postulate functional extensionality

Is funext really harmless?

$$funext : \forall (f\ g : A \to B).(\forall\, x.\, f\ x = g\ x) \to f = g$$

Is funext really harmless?

$$funext : \forall (f\ g : A \to B).(\forall x.\ f\ x = g\ x) \to f = g$$

- Lean includes funext with Mathlib
- Rocq and Agda carefully avoid funext in their standard libraries

Is funext really harmless?

$$funext : \forall (f\ g : A \to B).(\forall\, x.\, f\ x = g\ x) \to f = g$$

- Lean includes funext with Mathlib
- Rocq and Agda carefully avoid funext in their standard libraries

This is the good old constructive vs classical math debate, isn't it?

Is funext really harmless?

$$funext : \forall (f\ g : A \to B).(\forall x.f\ x = g\ x) \to f = g$$

- Lean includes funext with Mathlib
- Rocq and Agda carefully avoid funext in their standard libraries

This is the good old constructive vs classical math debate, isn't it?

Except that funext is constructive! So, what's the problem?

Is funext really harmless?

We can use funext to define noncanonical integers:
closed integers that do not evaluate to proper numerals

```
Axiom funext : forall {A B : Type}, forall (f g : A -> B),
    (forall x, f x = g x) -> f = g.

Definition noncanonical : nat :=
  eq_rect (fun x => x) (fun f => f nat) 0 (fun x => x)
          (funext _ _ (fun _ => eq_refl _)).

Eval compute in noncanonical.   (* not equal to a numeral *)
```

Is funext really harmless?

We can use funext to define noncanonical integers:
closed integers that do not evaluate to proper numerals

```
Axiom funext : forall {A B : Type}, forall (f g : A -> B),
    (forall x, f x = g x) -> f = g.

Definition noncanonical : nat :=
  eq_rect (fun x => x) (fun f => f nat) 0 (fun x => x)
          (funext _ _ (fun _ => eq_refl _)).

Eval compute in noncanonical.   (* not equal to a numeral *)
```

We can still evaluate these number by erasing `eq_rect`
(although it may lead to nontermination for open terms)

Is funext really harmless?

We can use funext to define noncanonical integers:
closed integers that do not evaluate to proper numerals

```
Axiom funext : forall {A B : Type}, forall (f g : A -> B),
    (forall x, f x = g x) -> f = g.

Definition noncanonical : nat :=
  eq_rect (fun x => x) (fun f => f nat) 0 (fun x => x)
          (funext _ _ (fun _ => eq_refl _)).

Eval compute in noncanonical.   (* not equal to a numeral *)
```

We can still evaluate these number by erasing `eq_rect`
(although it may lead to nontermination for open terms)

→ at this point, we might as well be doing realisability...

Setoids

To avoid funext, we can use setoids:

- A setoid is a type equipped with an equivalence relation $\sim$
- A setoid function is a function f that preserves the relation: $x \sim y \to f\ x \sim f\ y$

Setoids

To avoid funext, we can use setoids:

- A setoid is a type equipped with an equivalence relation $\sim$
- A setoid function is a function f that preserves the relation: $x \sim y \to f\, x \sim f\, y$

Some examples of setoids:

$\mathbb{N}$: elements are numbers, equality is trivial

Setoids

To avoid funext, we can use setoids:

- A setoid is a type equipped with an equivalence relation $\sim$
- A setoid function is a function f that preserves the relation:
  $x \sim y \to f\,x \sim f\,y$

Some examples of setoids:

$\mathbb{N}$: elements are numbers, equality is trivial
Prop : elements are propositions, equality is logical equivalence

Setoids

To avoid funext, we can use setoids:

- A setoid is a type equipped with an equivalence relation $\sim$
- A setoid function is a function f that preserves the relation: $x \sim y \to f\,x \sim f\,y$

Some examples of setoids:

$\mathbb{N}$	: elements are numbers, equality is trivial
$Prop$	: elements are propositions, equality is logical equivalence
$A \to B$	: elements are setoid functions, equality is pointwise

Problem solved?

Working with setoids lets us do extensional maths in Rocq

Problem solved?

Working with setoids lets us do extensional maths in Rocq

...well, more or less! Some problems:

Problem solved?

Working with setoids lets us do extensional maths in Rocq

...well, more or less! Some problems:

- Setoids are very limited:
  with these definitions, we only get HOL -- no dependent types

Problem solved?

Working with setoids lets us do extensional maths in Rocq

...well, more or less! Some problems:

- Setoids are very limited:
  with these definitions, we only get HOL -- no dependent types
- Setoids are awfully bureaucratic:
  we have to prove that every function respects setoid equalities
  (somewhat mitigated by Rocq's automation)

A Setoid compiler

Using setoids by hand feels like doing the job of a compiler
So, let's write a proper compiler to do it instead!

A Setoid compiler

Using setoids by hand feels like doing the job of a compiler
So, let's write a proper compiler to do it instead!

We want a compilation function $[\![_]\!]$ from syntax to syntax, such that

$$\Gamma \vdash t : A \quad \rightsquigarrow \quad [\![\Gamma]\!] \vdash [\![t]\!] : [\![A]\!]$$
$$\Gamma \vdash t \equiv u : A \quad \rightsquigarrow \quad [\![\Gamma]\!] \vdash [\![t]\!] \equiv [\![u]\!] : [\![A]\!]$$

A Setoid compiler

Using setoids by hand feels like doing the job of a compiler
So, let's write a proper compiler to do it instead!

We want a compilation function $[\![_]\!]$ from syntax to syntax, such that

$$\Gamma \vdash t : A \quad \rightsquigarrow \quad [\![\Gamma]\!] \vdash [\![t]\!] : [\![A]\!]$$
$$\Gamma \vdash t \equiv u : A \quad \rightsquigarrow \quad [\![\Gamma]\!] \vdash [\![t]\!] \equiv [\![u]\!] : [\![A]\!]$$

We also require that

- the translation of funext is provable in plain Rocq

A Setoid compiler

Using setoids by hand feels like doing the job of a compiler
So, let's write a proper compiler to do it instead!

We want a compilation function $[\![_]\!]$ from syntax to syntax, such that

$$\begin{aligned} \Gamma \vdash t : A &\quad\rightsquigarrow\quad [\![\Gamma]\!] \vdash [\![t]\!] : [\![A]\!] \\ \Gamma \vdash t \equiv u : A &\quad\rightsquigarrow\quad [\![\Gamma]\!] \vdash [\![t]\!] \equiv [\![u]\!] : [\![A]\!] \end{aligned}$$

We also require that

- the translation of funext is provable in plain Rocq
- the translation of $\mathbb{N}$ only contains canonical integers

A Setoid compiler

Using setoids by hand feels like doing the job of a compiler
So, let's write a proper compiler to do it instead!

We want a compilation function $[\![_]\!]$ from syntax to syntax, such that

$$\begin{aligned} \Gamma \vdash t : A \quad &\rightsquigarrow \quad [\![\Gamma]\!] \vdash [\![t]\!] : [\![A]\!] \\ \Gamma \vdash t \equiv u : A \quad &\rightsquigarrow \quad [\![\Gamma]\!] \vdash [\![t]\!] \equiv [\![u]\!] : [\![A]\!] \end{aligned}$$

We also require that

- the translation of funext is provable in plain Rocq
- the translation of $\mathbb{N}$ only contains canonical integers
- the translation of $\bot$ is not provable in plain Rocq

A Setoid compiler

This way, we can reason in Rocq's type theory with funext as a high-level language, and compile our work in plain Rocq afterwards

$$\vdash foo : \forall (a\ b\ c\ n : \mathbb{N}_{>0}).\ a^n + b^n = c^n \to n < 3$$

uses *funext*

$$\vdash [\![foo]\!] : [\![\forall (a\ b\ c\ n : \mathbb{N}_{>0}).\ a^n + b^n = c^n \to n < 3]\!]$$

uses $[\![funext]\!]$,
which is provable

A Setoid compiler

These ideas are due to Hofmann, who wrote two such "compilers"

A Setoid compiler

These ideas are due to Hofmann, who wrote two such "compilers"

Unfortunately, neither is truly satisfactory:

- The first one does not fully support Rocq's dependent types
- The second one fails to capture some definitional equalities

The missing ingredient

In 1999, Altenkirch adds definitional proof irrelevance to the mix

$$\frac{}{SProp : Type} \qquad \frac{P : SProp \qquad a\ b : P}{a \equiv b}$$

The missing ingredient

In 1999, Altenkirch adds definitional proof irrelevance to the mix

$$\frac{}{SProp : Type} \qquad \frac{P : SProp \qquad a\ b : P}{a \equiv b}$$

Altenkirch puts the setoid relations in $SProp$, so that they satisfy more definitional equalities

The missing ingredient

In 1999, Altenkirch adds definitional proof irrelevance to the mix

$$\frac{}{SProp : Type} \qquad \frac{P : SProp \qquad a\ b : P}{a \equiv b}$$

Altenkirch puts the setoid relations in $SProp$, so that they satisfy more definitional equalities

These extra equations are just enough to fulfill Hofmann's plan: a model of type theory in type theory which validates funext

Getting rid of the compiler

The setoid model lets us give computational content to funext

Getting rid of the compiler

The setoid model lets us give computational content to funext

Can we get rid of this complex compilation phase and work directly in a type theory that supports funext?

Getting rid of the compiler

The setoid model lets us give computational content to funext

Can we get rid of this complex compilation phase and work directly in a type theory that supports funext?

In 2007, Altenkirch, McBride & Swierstra propose to extend the language of type theory to reflect what happens in the setoid model

→ Observational Type Theory

Getting rid of the compiler

The setoid model lets us give computational content to funext

Can we get rid of this complex compilation phase and work directly in a type theory that supports funext?

In 2007, Altenkirch, McBride & Swierstra propose to extend the language of type theory to reflect what happens in the setoid model

→ Observational Type Theory

...but that is another story!

II.

Proof-relevant setoids

Let's get more technical

$$
\begin{aligned}
&\text{Record Setoid} :\equiv \{ \\
&\quad A \qquad\quad : \text{Type} \\
&\quad _ \sim_A _ : A \to A \to \text{SProp} \\
&\quad \text{refl} \quad : x \sim_A x \\
&\quad \text{sym} \quad : x \sim_A y \to y \sim_A x \\
&\quad \text{trans} \; : x \sim_A y \to y \sim_A z \to x \sim_A z \\
&\}
\end{aligned}
$$

Let's get more technical

$$
\begin{aligned}
&\text{Record Setoid} :\equiv \{\\
&\quad A \qquad\quad : \text{Type}\\
&\quad _ \sim_A _ : A \to A \to \text{SProp}\\
&\quad \text{refl} \quad : x \sim_A x\\
&\quad \text{sym} \quad : x \sim_A y \to y \sim_A x\\
&\quad \text{trans} \quad : x \sim_A y \to y \sim_A z \to x \sim_A z\\
&\}
\end{aligned}
$$



Several option for this universe

Let's get more technical

$$
\begin{aligned}
&\textbf{Record}\ \mathsf{Setoid} :\equiv \{\\
&\quad A \qquad\quad : \mathsf{Type}\\
&\quad _ \sim_A _ : A \to A \to \mathsf{Prop}\\
&\quad \mathsf{refl} \quad : x \sim_A x\\
&\quad \mathsf{sym} \quad\ : x \sim_A y \to y \sim_A x\\
&\quad \mathsf{trans} \ : x \sim_A y \to y \sim_A z \to x \sim_A z\\
&\}
\end{aligned}
$$



Let's get more technical

$$
\begin{aligned}
&\text{Record Setoid} :\equiv \{\\
&\quad A \qquad\quad : \text{Type}\\
&\quad _ \sim_A _ : A \to A \to \text{Type}\\
&\quad \text{refl} \quad : x \sim_A x\\
&\quad \text{sym} \quad : x \sim_A y \to y \sim_A x\\
&\quad \text{trans} \quad : x \sim_A y \to y \sim_A z \to x \sim_A z\\
&\}
\end{aligned}
$$



Several option for this universe

Let's get more technical

```
Record Setoid :≡ {
  A        : Type
  _ ∼A _   : A → A → Type
  refl     : x ∼A x
  sym      : x ∼A y → y ∼A x
  trans    : x ∼A y → y ∼A z → x ∼A z
}
```

Several option for this universe

This choice has a strong influence on the behaviour of setoids

Review of universes

Review of universes

SProp

Impredicative, definitionally proof-irrelevant, large elim only allowed for $\bot$

Review of universes

SProp

Impredicative, definitionally proof-irrelevant, large elim only allowed for $\bot$

★★★☆☆

Review of universes

SProp

Impredicative, definitionally proof-irrelevant, large elim only allowed for $\bot$ ★★★☆☆

Prop

Impredicative, morally proof-irrelevant, large elim only allowed for subsingletons (in particular, $\bot$, Id, Acc)

Review of universes

SProp

Impredicative, definitionally proof-irrelevant, large elim only allowed for $\bot$ ★★★☆☆

Prop

Impredicative, morally proof-irrelevant, large elim only allowed for subsingletons (in particular, $\bot$, Id, Acc) ★★★☆☆

Review of universes

SProp

Impredicative, definitionally proof-irrelevant, large elim only allowed for $\bot$ ★★★☆☆

Prop

Impredicative, morally proof-irrelevant, large elim only allowed for subsingletons (in particular, $\bot$, Id, Acc) ★★★☆☆

Impredicative Set

Impredicative, proof-relevant, weird, large elim only allowed for small inductives

Review of universes

SProp

Impredicative, definitionally proof-irrelevant, large elim only allowed for $\bot$ ★★★☆☆

Prop

Impredicative, morally proof-irrelevant, large elim only allowed for subsingletons (in particular, $\bot$, Id, Acc) ★★★☆☆

Impredicative Set

Impredicative, proof-relevant, weird, large elim only allowed for small inductives ★★☆☆☆

Review of universes

SProp
Impredicative, definitionally proof-irrelevant, large elim only allowed for $\bot$ ★★★☆☆

Prop
Impredicative, morally proof-irrelevant, large elim only allowed for subsingletons (in particular, $\bot$, Id, Acc) ★★★☆☆

Impredicative Set
Impredicative, proof-relevant, weird, large elim only allowed for small inductives ★★☆☆☆

Type
Predicative hierarchy, proof-relevant, large elim allowed

Review of universes

SProp
Impredicative, definitionally proof-irrelevant, large elim only allowed for $\bot$ ★★★☆☆

Prop
Impredicative, morally proof-irrelevant, large elim only allowed for subsingletons (in particular, $\bot$, Id, Acc) ★★★☆☆

Impredicative Set
Impredicative, proof-relevant, weird, large elim only allowed for small inductives ★★☆☆☆

Type
Predicative hierarchy, proof-relevant, large elim allowed ★★★★★

SProp setoids

SProp setoids are convenient since they satisfy more equations

SProp setoids

SProp setoids are convenient since they satisfy more equations

However it also means that we introduce a system with two layers:

- Computational data (the inhabitants of the setoids)
- Noncomputational data (the equality proofs)

SProp setoids

SProp setoids are convenient since they satisfy more equations

However it also means that we introduce a system with two layers:

- Computational data (the inhabitants of the setoids)
- Noncomputational data (the equality proofs)

The strong boundary between these two layers makes the setoids less powerful

SProp setoids

Example: when we have a relation $R : A \to A \to Type$ on a setoid, the setoid model lets us define the quotient setoid A/R

before doing so, we must turn the relation R into a $SProp$-valued relation $\|R\|$. This truncation is a lossy operation

SProp setoids

Example: when we have a relation $R : A \to A \to \mathit{Type}$ on a setoid, the setoid model lets us define the quotient setoid A/R

before doing so, we must turn the relation R into a SProp-valued relation $\|R\|$. This truncation is a lossy operation

Some unfortunate consequences:

- we cannot interpret countable/dependent choice
- functional graphs and functions are different
- we can prove that System F terminates, but not write an evaluator

Type setoids

Using Type means that the truncation barely does anything:
we simply change the equality relation to the trivial one.

The boundary between types and propositions is very thin, which lets us interpret some new principles:

Type setoids

Using Type means that the truncation barely does anything:
we simply change the equality relation to the trivial one.

The boundary between types and propositions is very thin, which lets us interpret some new principles:

Function comprehension / Unique choice
Functional relations are the same thing as functions

Type setoids

Using Type means that the truncation barely does anything:
we simply change the equality relation to the trivial one.

The boundary between types and propositions is very thin, which lets us interpret some new principles:

Function comprehension / Unique choice

Functional relations are the same thing as functions

Countable and dependent choice

Since the setoid equality on $\mathbb{N}$ coincides with the meta-equality, every function out of $\mathbb{N}$ is automatically a setoid morphism.

Type setoids

Downsides:

- predicative → we cannot prove that System F normalises
- less equations → Hofmann's setoid model does not capture the equational theory or Rocq

Prop setoids

Using Prop is somewhere inbetween:
Truncation is a lossy operation, but we can recover some information

Prop setoids

Using Prop is somewhere inbetween:
Truncation is a lossy operation, but we can recover some information

Magic trick: well-founded induction using Acc

If we prove that a relation is well-founded in *Prop*, we can do induction on this relation to define objects in *Type*

Prop setoids

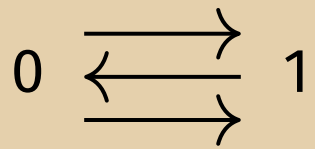
- we can write an evaluator for System F
- however, functional relations are not the same as functions
- not enough equations in Hofmann's setoid model

A fresh look at the setoid model

What if... we use *Type*/*Prop*-valued setoids, but still somehow find a way to get the setoid model equations out of *SProp*?

Strict presheaves

If we ask setoid functions to preserve reflexivity proofs, the category of setoids can be described as a category of fibrant presheaves



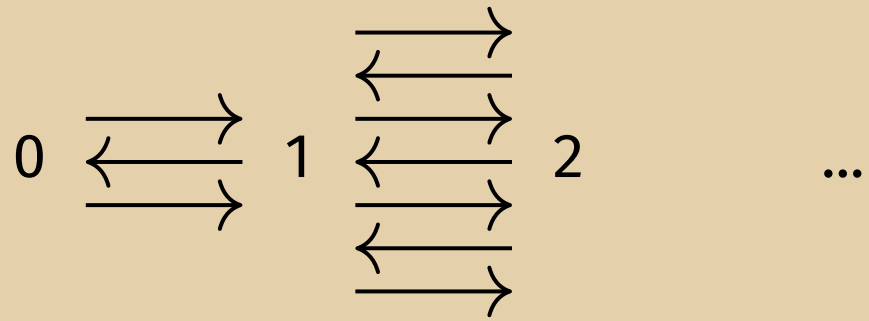
Pédrot 2019: in a theory with *SProp* and a strengthened *J* rule, we can do presheaf models of type theory in type theory

Combine this with a recent definition of universes for proof-relevant setoids (P. 2025), and we can get a fully functional setoid model with *Type* equalities!

Strict presheaves

However, asking setoid functions to preserve reflexivity is not so innocuous, and we cannot interpret unique choice any more.

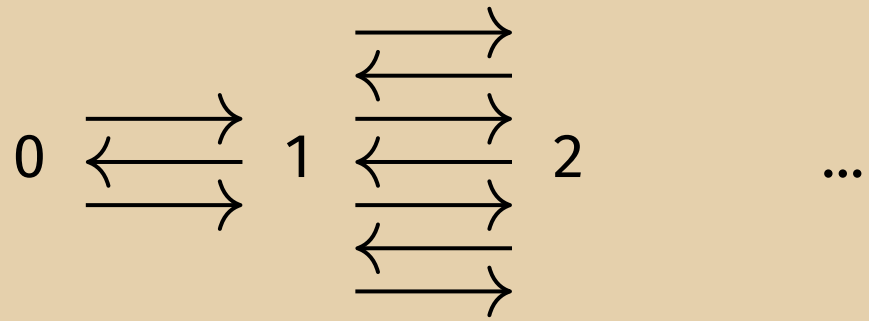
For this, we would need to go all the way to fibrant cubical sets



Strict presheaves

However, asking setoid functions to preserve reflexivity is not so innocuous, and we cannot interpret unique choice any more.

For this, we would need to go all the way to fibrant cubical sets



Good compromise for this new model: Prop valued setoids?

Conclusion

We have an improved version of Hofmann Altenkirch's setoid model, which supports some choice principles.

Can we get an improved version of observational type theory too?

Thank you!